

Experiment No. 1

Comparative Study of Low-Code Development Versus Traditional Application Development Approaches

1. Aim

To study and compare low-code development and traditional application development approaches based on their development process, coding requirements, development time, cost, flexibility, and maintenance.

2. Introduction to Traditional Application

Traditional application development is the conventional method of developing software applications in which developers write source code using programming languages, frameworks, libraries, and database technologies. Developers have complete control over the application's user interface, backend logic, database, security, and functionality.

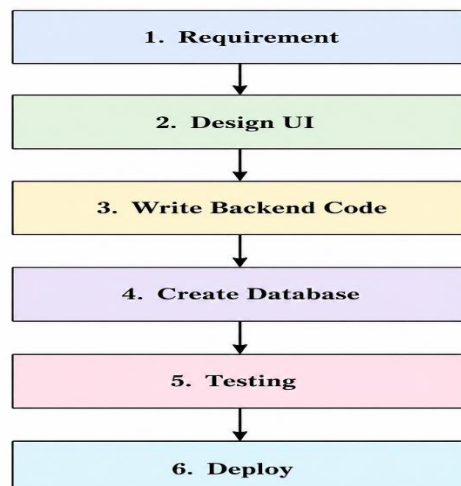
This approach requires good programming knowledge and technical expertise. Technologies such as Java, Python, JavaScript, HTML, CSS, SQL, and various frameworks can be used to develop applications. Since most components are developed and integrated manually, traditional development generally requires more time and effort. However, it provides high flexibility and customization, making it suitable for complex and large-scale applications.

3. Introduction to Low-Code

Low-code development is a modern approach to application development that allows applications to be created with minimal manual coding. It uses visual development tools, drag-and-drop components, predefined templates, reusable modules, and workflows to simplify the development process.

Low-code platforms allow developers to create forms, manage data, generate reports, automate workflows, and publish applications with less coding. This reduces development time and makes application development easier and faster. Low-code is particularly useful for business applications, workflow automation, prototypes, and applications that need to be developed quickly.

4. Diagram for Stages in Traditional Approach



5. Diagram for Steps in Low-Code



6. Comparative Analysis of Traditional and Low-Code Approach

Parameter	Traditional Approach	Low-Code Approach
Development Approach	Applications are developed mainly by writing code manually. Developers build and integrate the UI, backend, database, and other components.	Applications are developed mainly using visual interfaces and pre-built components. Manual coding is required only when additional customization is needed.
Coding Requirement	Requires extensive coding using programming languages and frameworks. Skilled developers are generally required to build the application.	Requires significantly less manual coding. Drag-and-drop tools, templates, and reusable components simplify development.
Development Time	Development usually takes more time because components need to be designed, coded, integrated, and tested separately.	Development is generally faster because many components and functionalities are already provided by the platform.
Customization	Provides a high level of customization and control. Developers can modify almost every part of the application according to requirements.	Customization is easier for standard requirements but may be limited by the features and capabilities provided by the low-code platform.
Technical Expertise	Requires good knowledge of programming languages, databases, APIs, frameworks, and software development concepts.	Requires comparatively less programming knowledge. Visual development tools allow applications to be created with less technical expertise.

Parameter	Traditional Approach	Low-Code Approach
Cost	Development cost can be higher because it requires skilled developers, longer development time, testing, and maintenance.	Development effort and time can be reduced, although organizations may need to pay licensing or subscription costs for the platform.
Deployment & Maintenance	Deployment may require server configuration and other technical steps. Updates often require modifying, testing, and redeploying the code.	Platforms generally simplify publishing and deployment. Changes can often be made visually and published with fewer manual steps.
Suitable Applications	Best suited for large, complex, performance-sensitive, or highly customized applications requiring complete control.	Best suited for business applications, forms, workflow automation, prototypes, and applications that need to be developed quickly.

7. Result

Thus, the traditional and low-code application development approaches were studied and compared. Traditional development provides greater flexibility, customization, and control but requires more coding, technical expertise, and development time. Low-code development reduces manual coding and enables faster application development using visual tools and reusable components. Therefore, the appropriate development approach can be selected based on the application requirements, complexity, development time, budget, and level of customization required.